

Contents

AI Agent Knowledge Base for a Financial Company

1. Purpose

2. Requirements

2.1 Functional

2.2 Security, Compliance, and Governance

2.3 Non-Functional

2.4 Explicit Non-Goals for v1

3. Design Principles

4. Proposed Solution Architecture

4.1 Storage of Originals and Derived Representations

4.2 Ingestion Pipeline & Bulk-Load Optimization

4.3 Data Model (Core Tables & Indexing)

4.4 Chunking Strategy

4.5 Adaptive Hybrid Retrieval Workflow

4.6 Answer Generation & Citation Enforcement

4.7 Evaluation and Quality Governance

4.8 Backup, Recovery, and Business Continuity

5. Technology Choices

6. Sizing, Performance, and Operational Notes

7. Scaling Gates (When to Evolve)

8. Delivery Roadmap

9. Risk Matrix & Mitigations

10. Summary

1

1

2

2

2

3

3

4

4

5

6

8

8

9

10

10

13

14

14

15

15

16

AI Agent Knowledge Base for a Financial Company

Requirements and Proposed Solution Architecture

Status: Draft v4 (pending approval) · Date: 2026-08-27

Change log:

- v4 adds a formal backup and recovery process (new §4.8): PostgreSQL PITR backups, immutable audit-log exports, backup immutability, restore ordering between database and document volume, RPO/RTO targets, and scheduled restore testing. Requirement N9, technology row, roadmap items, and risk rows added accordingly.
- v3 replaces S3/object storage with mounted block/file volumes (e.g. AWS EBS, on-prem NAS/SAN) for originals and derivatives.

1. Purpose

Build a retrieval-augmented AI agent that answers questions from a corpus of millions of internal financial and legal documents (text, PDF, Word, PowerPoint, spreadsheets, scanned files) while strictly enforcing relational access controls, producing verifiable citations, and maintaining a tamper-evident audit trail suitable for a regulated environment.

This architecture deliberately balances enterprise compliance with pragmatic simplicity: it reuses the team's established PostgreSQL operational expertise, avoids early multi-cluster sprawl (such as standalone vector databases, distributed search clusters, or external message brokers), and defines explicit quantitative gates for when specialized infrastructure becomes justified.

2. Requirements

2.1 Functional

ID	Requirement
F1	Ingest documents from multiple enterprise sources (file shares, SharePoint, email archives, document management systems) in PDF, DOCX, PPTX, XLSX, TXT, HTML, Markdown, and common scanned image formats.
F2	Handle scanned documents via OCR, with extraction confidence scored and attached at the chunk/page level.
F3	Preserve document hierarchy: section headers, page and slide numbers, table schemas, footnotes, and speaker notes.
F4	Detect and handle exact duplicates, near-duplicates, and superseded versions with alias mapping.
F5	Support hybrid retrieval — exact/lexical match (tickers, ISINs, clause numbers, dates, party names) and semantic match (conceptual questions, thematic synthesis, paraphrase).
F6	Filter by rich metadata: source, business unit, document type, date range, classification, and language.
F7	Return answers with verifiable citations at the page / slide / section / table level, deterministically linked to the exact document version used.
F8	Abstain or flag uncertainty when evidence is weak, contradictory, or superseded.
F9	Expose retrieval as an independent, secured REST/gRPC service consumable by the agent and other internal systems.
F10	Re-index automatically upon source modification; tombstone deleted content promptly.
F11	Provide a human-browsable, normalized Markdown corpus for domain expert QA, curation, and compliance audits.

2.2 Security, Compliance, and Governance

ID	Requirement
S1	Access control strictly mirrors source-system entitlements. ACL filters are applied pre-ranking at the database layer; unauthorized users can never retrieve or observe chunks from restricted documents.
S2	Information barriers (ethical walls, client/deal segregation) enforced at the tenant / collection partition level.
S3	Automated classification tags at ingestion: PII, MNPI, client-confidential, regulatory, and retention category.
S4	Full audit log per query: user identity, evaluated roles/groups, applied filters, retrieved chunk IDs, ranking scores, model/prompt versions, generated response, and rendered citations.
S5	Complete data residency: originals, derivatives, vector indices, and audit logs remain within the firm's approved security boundary. Models (embeddings, rerankers, LLMs) are self-hosted or private enterprise endpoints.
S6	Release v1 is strictly read-only : no execution of financial transactions, external communications, or automated decisions.
S7	Prompts, retrieved contexts, and outputs are archived as immutable business records under compliance supervision rules. Immutability is achieved by exporting closed audit-log partitions to write-once storage (see §4.8.3).

2.3 Non-Functional

ID	Requirement
N1	Corpus Scale: Low single-digit millions of source documents; estimated 20–30 million indexed chunks after deduplication and junk exclusion.
N2	Query Latency: p95 retrieval latency under 1.5 seconds (excluding LLM generation time).
N3	Ingestion Throughput: Initial backfill achievable within weeks; incremental sync within minutes of source document updates.
N4	Operational Simplicity: Minimize stateful operational dependencies. Zero multi-cluster maintenance overhead for v1.
N5	Reliability & Idempotency: Job processing is transactional, observable, and crash-resilient with automatic retry handling.
N6	Measurable Quality: Retrieval recall@20, citation accuracy, groundedness, and permission-leak rate continuously evaluated against a curated golden dataset.
N7	Portability: Chunk indices are disposable and 100% reconstructible from derivatives stored on the document volume without re-parsing raw files.
N8	Storage Platform: Originals and derivatives live on mounted block/file volumes (AWS EBS, on-prem NAS/SAN) inside the firm's network — no object-store dependency.
N9	Backup & Recovery: Every irreplaceable state store (document volume, PostgreSQL registry/ACL/audit data, model and prompt artifacts) is backed up on a defined schedule with RPO ≤ 1 hour and RTO ≤ 8 hours for full-service restore (targets to be confirmed with Business Continuity). Backups are immutable for their retention period, stay inside the firm's security boundary (S5), and are exercised by automated restore tests at least monthly.

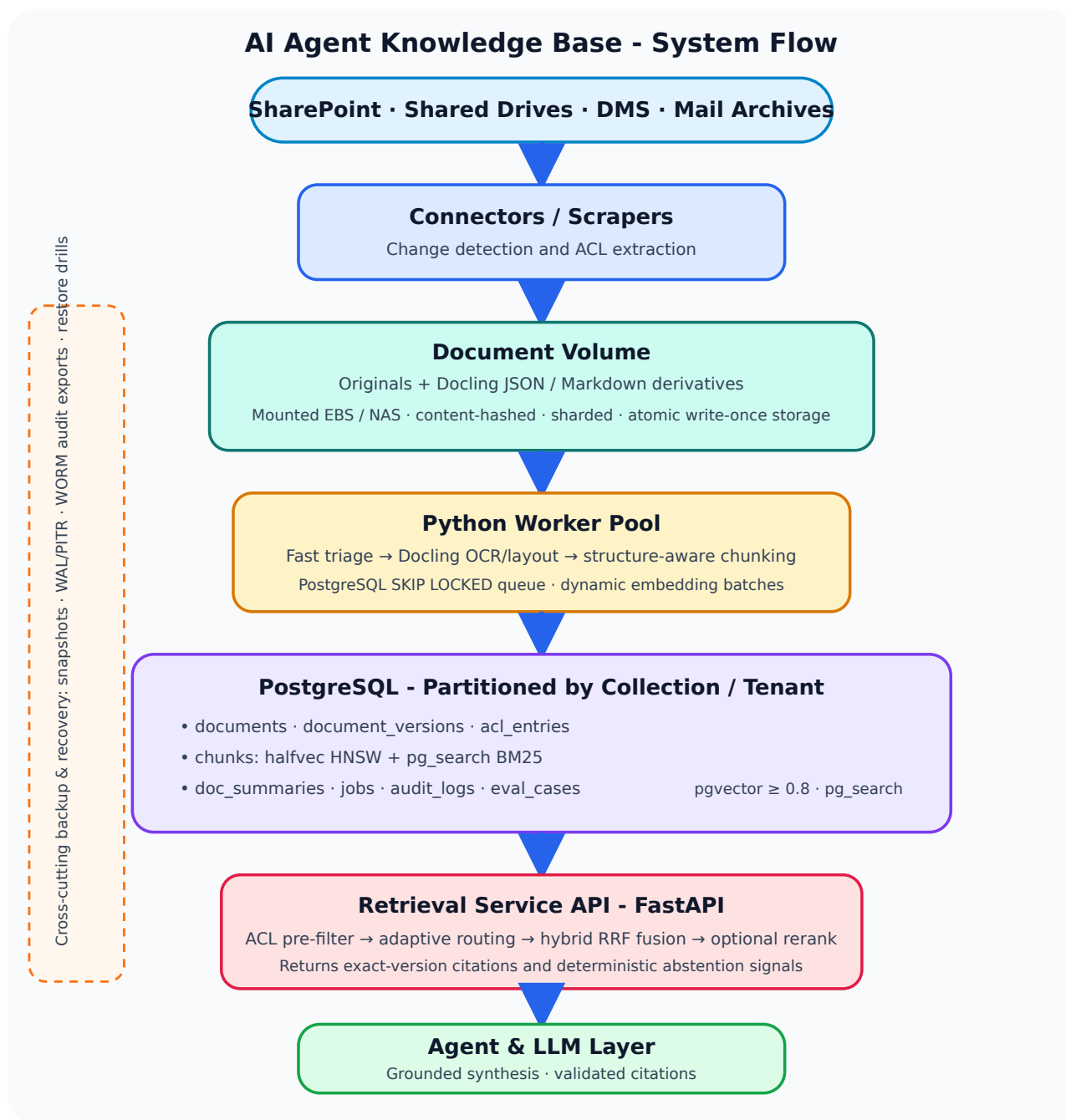
2.4 Explicit Non-Goals for v1

- Multi-step autonomous agent workflows or multi-turn tool execution beyond single-shot retrieval.
- Supervised fine-tuning (SFT) of foundation LLMs.
- Real-time streaming market data or sub-second event ingestion.
- Cross-lingual retrieval (focusing on English-language financial and legal corpora for v1).

3. Design Principles

1. **Documents are not chunks.** A document possesses lifecycle, lineage, access policies, and legal retention. A chunk is merely an ephemeral search artifact.
2. **Parse once, index many times.** Store normalized structured representations (Docling JSON + Markdown) in immutable storage. Re-chunking or re-embedding never triggers raw file re-parsing.
3. **One system of record.** PostgreSQL acts as the single transactional source of truth for document registries, entitlements, ingestion queues, search indices, and query audit trails.
4. **Adaptive two-tier retrieval.** Route broad thematic queries through document-level summaries to shrink search spaces, but allow targeted identifier/clause searches direct access to the chunk index to avoid missing critical needle-in-a-haystack terms.
5. **Protect relational performance during bulk load.** Separate bulk backfill operations (unindexed ingestion followed by batch index build) from ongoing incremental updates.
6. **Add components only when evaluation demands it.** Defer standalone vector databases or specialized search clusters until explicit scaling gates are breached.

4. Proposed Solution Architecture

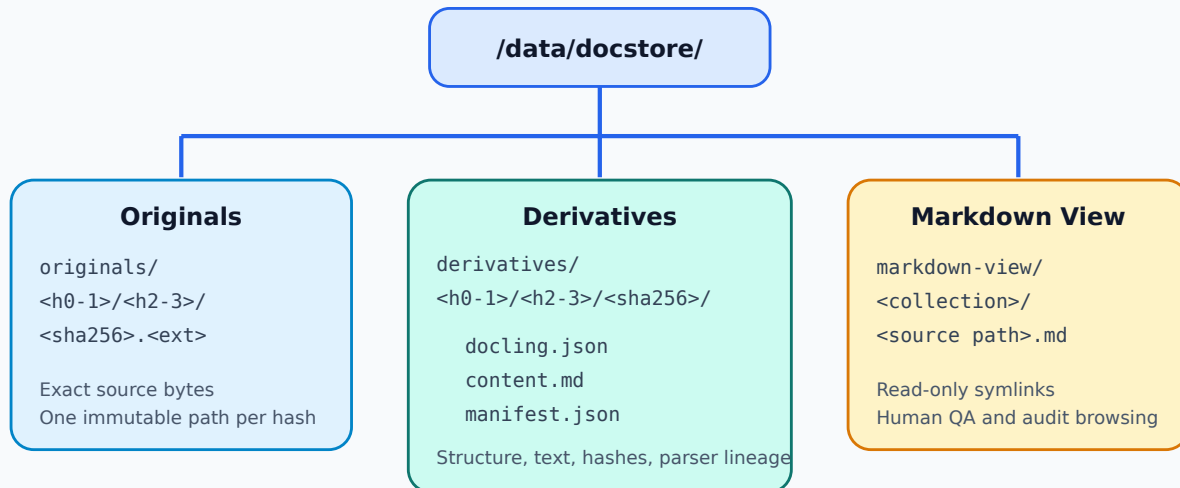


4.1 Storage of Originals and Derived Representations

Originals and derivatives are stored on **mounted block/file volumes** rather than an object store. On AWS this is EBS (gp3 or io2) attached to a storage node; on-premises it is a NAS/SAN export. The database never stores binary payloads; it stores a `storage_uri` pointing into the volume.

Volume layout (content-addressed, sharded):

Document Volume - Content-Addressed and Sharded



First four SHA-256 hex characters keep every directory bounded

- **Directory sharding** by the first four hex characters of the SHA-256 keeps every directory under a few thousand entries; a flat directory with millions of files degrades listing and metadata operations on every filesystem.
- **Filesystem:** XFS (preferred for millions of small files and large volumes) or ext4. LVM across multiple EBS volumes allows capacity growth without downtime.
- **Write-once discipline:** Workers write to a temp path on the same filesystem, `fsync`, then atomically `rename()` into place. A path is never overwritten; a changed document produces a new hash and a new path. Optionally apply `chattr +i` after rename so even a privileged process cannot modify the file in place.
- **Shared access for the worker pool:** EBS is single-attach, so the volume is mounted on one storage node and exported to workers over NFS v4 (read/write for ingestion workers, read-only for the retrieval/Markdown-view hosts). If the firm permits it, a managed NFS service (AWS EFS / FSx) can replace the storage node with the same directory layout; on-prem, the NAS export plays the same role. Docling and PyMuPDF workers read via local POSIX I/O, which is simpler and faster than object-store GETs.
- **Durability and recovery:** Automated EBS snapshots (hourly during backfill, daily thereafter, retained per the records policy) plus a periodic **scrub job** that re-hashes a sample of files and compares against `document_versions.content_hash`, flagging any corruption. For on-prem NAS, use the array's snapshot and replication features. Snapshot to a second Availability Zone or site for disaster recovery. The full backup schedule, immutability controls, and the restore procedure that keeps this volume consistent with PostgreSQL are defined in §4.8.
- **Capacity estimate:** ~3M originals at ~1 MB average ≈ 3 TB; Docling JSON and Markdown add roughly 1–2 TB. Provision 8–10 TB initially with LVM headroom; gp3 supports up to 16 TiB per volume.
- **Browsable Human View:** The `markdown-view/` tree exposes the normalized Markdown in an Obsidian-compatible structure so legal, compliance, and domain experts can inspect extracted tables and text during evaluation and auditing without touching database tables. It is a read-only view built from symlinks; it holds no data of its own.

4.2 Ingestion Pipeline & Bulk-Load Optimization

Each ingestion task is an idempotent step recorded in the transactional jobs table:

1. **Discover & Capture ACL:** Connectors poll or receive webhooks from source systems, capturing file metadata, modification timestamps, and access control lists (user/group SIDs).

2. **Fetch & Hash:** Stream the file to a temp path on the document volume while computing SHA-256. If the hash already exists in `document_versions`, discard the temp file, link metadata and terminate early; otherwise atomically rename it into `originals/...` and record `original_uri`.
 3. **Format-Aware Triage:**
 - **Fast-Path (Born-Digital PDFs / Plain Text):** Processed via PyMuPDF / fast extractors in milliseconds.
 - **Heavy-Path (Scans / Complex Multi-Column Layouts / Decks):** Dispatched to GPU worker pool running Docling with layout analysis and OCR.
 4. **Deduplication & Canonical Mapping:** Near-duplicate detection via MinHash/SimHash on normalized text. Aliases point to the canonical document version.
 5. **Structure-Aware Chunking:** Chunk based on structural document boundaries (see Section 4.4).
 6. **Self-Hosted Embedding Generation:** Compute embeddings on local GPU workers in dynamic batches.
 7. **Database Ingestion & Indexing Lifecycle:**
 - **Initial Backfill (Millions of chunks):** Chunks and vectors are inserted directly into unindexed PostgreSQL staging tables. HNSW and BM25 indices are constructed in a single parallelized batch build once data loading is complete, eliminating massive index churn and write amplification.
 - **Incremental Mode (Live sync):** Chunks are inserted directly into indexed partition tables with immediate index updates.
-

4.3 Data Model (Core Tables & Indexing)

-- Core document registry

```
CREATE TABLE documents (
  document_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  source_system    TEXT NOT NULL,
  source_path      TEXT NOT NULL,
  canonical_hash   TEXT NOT NULL,
  doc_type         TEXT NOT NULL,
  collection_id    TEXT NOT NULL,
  classification   TEXT[] DEFAULT '{}',
  created_at       TIMESTAMPTZ DEFAULT clock_timestamp()
);
```

-- Version tracking and lineage

```
CREATE TABLE document_versions (
  document_version_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id         UUID NOT NULL REFERENCES documents(document_id) ON DELETE CASCADE,
  content_hash        TEXT NOT NULL,
  source_version_id   TEXT,
  original_uri        TEXT NOT NULL, -- e.g. file:///data/docstore/originals/ab/cd/<sha256>.pdf
  derivative_uri      TEXT NOT NULL, -- e.g. file:///data/docstore/derivatives/ab/cd/<sha256>/
  original_bytes      BIGINT,
  ocr_confidence      REAL,
  parser_version      TEXT NOT NULL,
  is_current          BOOLEAN DEFAULT TRUE,
  parsed_at           TIMESTAMPTZ DEFAULT clock_timestamp(),
  tombstoned_at       TIMESTAMPTZ
);
```

-- Pre-filtered Entitlements

```
CREATE TABLE acl_entries (
```

```

    document_id          UUID NOT NULL REFERENCES documents(document_id) ON DELETE CASCADE,
    principal_id          TEXT NOT NULL, -- User SID, AD Group, or Role
    principal_type        TEXT NOT NULL, -- 'user', 'group', 'tenant'
    PRIMARY KEY (document_id, principal_id)
);

CREATE INDEX idx_acl_principal ON acl_entries (principal_id);

-- Document-level summaries for two-tier retrieval
CREATE TABLE doc_summaries (
    document_id          UUID PRIMARY KEY REFERENCES documents(document_id) ON DELETE CASCADE,
    collection_id        TEXT NOT NULL,
    summary_text         TEXT NOT NULL,
    tsv                  tsvector,
    emb                  halfvec(1024)
);

-- Chunk storage (Partitioned by collection/business unit)
CREATE TABLE chunks (
    chunk_id             UUID DEFAULT gen_random_uuid(),
    document_version_id  UUID NOT NULL REFERENCES document_versions(document_version_id) ON DELETE CASCADE,
    collection_id        TEXT NOT NULL,
    chunk_index          INTEGER NOT NULL,
    heading_path         TEXT,
    page_start           INTEGER,
    page_end             INTEGER,
    slide_no            INTEGER,
    table_ref            TEXT,
    chunk_text           TEXT NOT NULL,
    emb                  halfvec(1024),
    embedding_model      TEXT NOT NULL,
    is_active            BOOLEAN DEFAULT TRUE,
    PRIMARY KEY (chunk_id, collection_id)
) PARTITION BY LIST (collection_id);

-- Operational Job Queue (Partitioned or Unlogged Staging for Bulk Ingest)
CREATE TABLE ingestion_jobs (
    job_id              BIGSERIAL PRIMARY KEY,
    stage               TEXT NOT NULL,
    payload             JSONB NOT NULL,
    status              TEXT NOT NULL DEFAULT 'pending',
    attempts            INTEGER DEFAULT 0,
    locked_at           TIMESTAMPTZ,
    created_at          TIMESTAMPTZ DEFAULT clock_timestamp()
);

-- Audit and Supervision Log
CREATE TABLE query_audit_logs (
    query_id            UUID DEFAULT gen_random_uuid(),
    user_id             TEXT NOT NULL,
    user_roles          TEXT[] NOT NULL,
    query_text          TEXT NOT NULL,
    applied_filters     JSONB NOT NULL,
    retrieved_chunk_ids UUID[] NOT NULL,
    retrieval_scores    REAL[] NOT NULL,
    model_version       TEXT NOT NULL,

```

```

prompt_version    TEXT NOT NULL,
generated_answer  TEXT NOT NULL,
citations         JSONB NOT NULL,
latency_ms       INTEGER NOT NULL,
created_at        TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp(),
PRIMARY KEY (query_id, created_at)
) PARTITION BY RANGE (created_at); -- monthly partitions; closed partitions exported to WORM (§4.8.3)

```

Indexing Strategy:

- **Vector Search:** HNSW index on `chunks.emb` using `halfvec_cosine_ops` (e.g., `m=16`, `ef_construction=128`).
- **Lexical Search:** `pg_search` (BM25 extension) or GIN index on `chunk_text` for BM25 term weighting and document length normalization.
- **Relational Metadata:** Composite B-tree on `(collection_id, is_active)`.

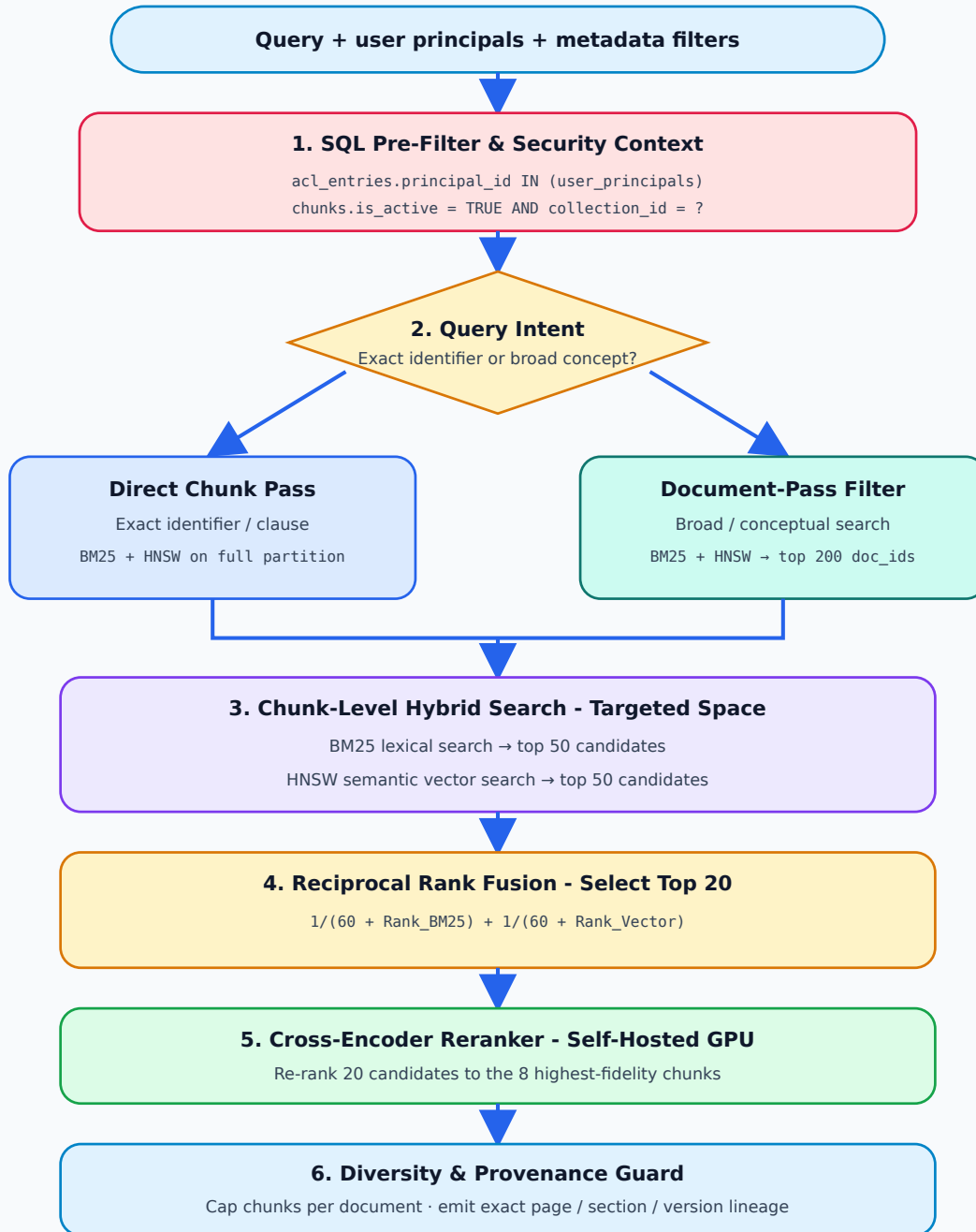
4.4 Chunking Strategy

Document Type	Chunking Boundary	Context Enrichment
Narrative Docs (PDF / Word)	Heading boundaries; split at 400–800 tokens on clean paragraph breaks	Prepend complete heading hierarchy path (e.g., <code>Credit Agreement > Section 4.2 > Representations</code>); retain footnotes inline.
Slide Presentations (PPTX / PDF)	One slide per chunk	Include presentation title, slide title, body text, and presenter notes.
Tables & Financial Schedules	Single table or coherent row-group block	Retain header row for all row chunks; attach raw structured JSON in metadata for deterministic numeric query lookup.
Spreadsheets (XLSX)	Named range or logical grid region	Header row and column serialization; skip uncaptioned pure numeric matrix noise.
Scanned Documentation	Same as Narrative Docs	Attach page-level OCR confidence score; flag chunks below threshold (<85%) in prompt and citations.

4.5 Adaptive Hybrid Retrieval Workflow

To prevent false negatives in specific clause searches while keeping broad queries lightning-fast, the retrieval engine uses an **Adaptive Hybrid Strategy**:

Adaptive Hybrid Retrieval



4.6 Answer Generation & Citation Enforcement

- **Context Assembly:** The LLM receives strictly the top re-ranked chunks with metadata headers ([DocID: XYZ, Page: 14, Version: 2]).
- **Grounded Prompting:**
 - Every factual claim must be immediately accompanied by its corresponding citation token.
 - If retrieved passages are contradictory, the model explicitly highlights the discrepancy across document versions.
 - If insufficient context exists, the model must explicitly respond: "The available documentation does not contain sufficient evidence to answer this inquiry."

- **Audit Traceability:** Query string, retrieved chunks, generated text, and citations are recorded in `query_audit_logs`.

4.7 Evaluation and Quality Governance

- **Golden Test Suite:** 300+ enterprise test queries curated across Legal, Risk, Credit, and Operations teams.
- **Evaluation Criteria:**
 - **Permission-Leak Rate:** 0.00% tolerance (systematic adversarial security test cases).
 - **Recall@20:** $\geq 88\%$ target on golden retrieval dataset.
 - **Citation Precision & Faithfulness:** $\geq 92\%$ automated evaluation verified by spot-checks.
 - **Latency SLA:** Retrieval p95 $\leq 1.5s$; End-to-end response p95 $\leq 4.5s$.

4.8 Backup, Recovery, and Business Continuity

Volume snapshots alone are not a backup strategy for this system. PostgreSQL is the system of record (Principle 3): without `documents`, `document_versions`, and `acl_entries`, the content-addressed files on the volume are an anonymous pile of hashes with no entitlements, no lineage, and no citations. And the audit log is a regulated business record (S7), not operational telemetry. This section defines what is backed up, how, how it is restored, and how we prove that restores work.

4.8.1 What Must Be Backed Up (Data Tiers)

Tier	Data	Replaceable?	Backup Approach
1 — Irreplaceable, regulated	<code>query_audit_logs</code> ; prompt/model version registry	No. Loss is a compliance finding.	Postgres PITR plus daily export of closed partitions to write-once (WORM) storage; retained per supervision/records policy (typically 5–7 years).
1 — Irreplaceable, structural	<code>documents</code> , <code>document_versions</code> , <code>acl_entries</code> , <code>doc_summaries</code> , <code>eval_cases</code> ; originals and Docling derivatives on the document volume	Originals could in theory be re-fetched from source systems, but sources purge, versions get superseded, and re-parsing 3M files takes weeks. Treat as irreplaceable.	Postgres PITR; volume snapshots with cross-AZ/site copy and snapshot lock.
2 — Reconstructible but expensive	chunks (text + halfvec embeddings + HNSW/BM25 indices)	Yes, from derivatives (N7) — but a full re-chunk + re-embed of 20–25M chunks is measured in days of GPU time, which blows the RTO.	Included in Postgres PITR by default. Backup size (~50 GB vectors + indices) is acceptable. May be excluded from the WORM export.
3 — Ephemeral	<code>ingestion_jobs</code> , staging tables, <code>markdown-view/</code> symlinks	Yes, trivially.	Not separately backed up. After restore, connectors re-run change detection and idempotent jobs resume (N5); symlink tree is regenerated.

Tier	Data	Replaceable?	Backup Approach
Config & artifacts	Schema migrations, connector configs, chunking rules, prompt templates, embedding/reranker model weights, Docling/parser versions, golden dataset	Partially — but a citation issued with <code>embedding_model = X</code> and <code>prompt_version = Y</code> can only be reproduced if X and Y are retained.	Git for code/config/prompts; model weights and parser container images in an internal artifact registry that is itself backed up. Never rely on re-downloading a public model checkpoint that may be revised or withdrawn.

4.8.2 PostgreSQL Backups

- **Continuous WAL archiving** to a separate volume/bucket-equivalent inside the security boundary, giving point-in-time recovery to any second. This is what delivers the 1-hour RPO; a nightly dump alone would mean up to 24 hours of lost ACL changes and audit records.
- **Base backups:** weekly full + daily incremental using **pgBackRest** (self-managed) or the platform's automated backups with PITR enabled (RDS/Aurora). **pgBackRest** is preferred for self-hosted because it supports parallel backup/restore of multi-hundred-GB clusters, block-level incrementals, and backup-set verification.
- **Retention:** 35 days of PITR-capable backups online; monthly full backups retained 13 months; audit-log WORM exports retained per records policy (see 4.8.3). Retention is enforced by policy on the backup repository, not by an ad-hoc cron job.
- **High availability is not backup.** A streaming standby replica (synchronous within the region, or asynchronous cross-AZ) is deployed for failover and for offloading backup I/O, but a replica faithfully replicates a `DROP TABLE` or a bad ACL sync within milliseconds. PITR backups exist precisely for the cases replication cannot cover: logical corruption, operator error, and malicious action.
- **Bulk backfill mode (Phase 3):** WAL volume during the unindexed bulk load will be very large. Take a full base backup immediately before the load and immediately after the batch index build; during the load itself, WAL archiving continues but retention may be temporarily relaxed for the staging tables only. `UNLOGGED` staging tables (which do not generate WAL) are acceptable **only** because their contents are re-derivable from the volume; they must be converted to logged tables or copied into logged partitions before the load is declared complete.
- **Encryption and residency:** Backups are encrypted at rest with firm-managed keys and never leave the approved boundary (S5). Backup storage is in a separate account/project or on a separate array from the primary database so that a single credential compromise cannot destroy both.

4.8.3 Immutable Audit-Log Archive (S7)

- `query_audit_logs` is partitioned by month (`PARTITION BY RANGE (created_at)`). This is a small change to the DDL in §4.3 and also keeps the table's indexes manageable at production query volumes.
- A nightly job exports every **closed** partition (i.e. the previous month, once the month has rolled over) plus a daily rolling export of the current month to a **write-once archive**: a dedicated volume with `chattr +i` applied post-write and locked snapshots, or the firm's existing supervision/records-management archive if one exists. Each export file carries a SHA-256 manifest that is itself recorded in a separate ledger table.
- Exports are in a self-describing, tool-independent format (Parquet or gzipped JSON Lines) so that they remain readable after the Postgres schema evolves.
- Retrieved chunk text is included in the export, not only the chunk IDs. Chunk IDs are stable, but a future re-chunk or re-embed (embedding model upgrade, §9) may retire them; the archived record must be self-contained enough to reconstruct what the user actually saw.
- Deletion of archived audit records is by records-retention policy only, executed through the records-management process, never by an engineering job.

4.8.4 Document Volume Backups The snapshot schedule from §4.1 is retained and tightened:

- **EBS:** Hourly snapshots during backfill and daily thereafter, managed by a data-lifecycle policy. Every snapshot is **copied to a second AZ or region** within the firm's boundary and protected with **snapshot lock / recycle-bin retention** so that neither a compromised administrator nor an errant automation can delete backups inside the retention window. Retain 30 daily, 12 monthly, and annual snapshots per records policy.
- **On-prem NAS/SAN:** Array-native scheduled snapshots with the same cadence, replicated to the DR site, with snapshot immutability enabled where the array supports it.
- **Application-level consistency:** Because the volume is write-once and content-addressed, a snapshot taken at any instant is consistent by construction — a partially written file lives only under the temp path and is never referenced by the database. No quiescing is needed.
- **Scrub job** (§4.1) runs weekly on a random 1% sample and monthly on any file referenced by a `document_versions` row with `is_current = TRUE`; mismatches are quarantined, alerted, and restored from the most recent snapshot that verifies.

4.8.5 Restore Ordering and Database-Volume Consistency The database and the volume are backed up independently, so a restore must be sequenced to avoid dangling `original_uri` / `derivative_uri` references:

1. **Restore PostgreSQL first** to the chosen recovery point T (PITR).
2. **Restore the document volume from a snapshot taken at or after T.** Because paths are never overwritten and a changed document always produces a new hash, a newer volume snapshot can only contain extra files (harmless orphans); an older snapshot could be missing files the database references. Newer is always safe; older never is.
3. **Run the reconciliation job:** for every `document_versions` row, verify that `original_uri` and `derivative_uri` exist on the volume and that the file hash matches `content_hash`. Missing or mismatched entries are queued as `refetch` jobs to the connectors (idempotent by design, §4.2). Orphan files on the volume with no database row are listed and left in place for the retention window, then garbage-collected.
4. **Re-run connector change detection** from the last successful sync timestamp stored in the restored database, so that documents and ACL changes that occurred after T are re-ingested. ACL re-sync is mandatory before the retrieval API is reopened to users — serving stale entitlements is a permission-leak scenario (S1).
5. **Regenerate** `markdown-view/` symlinks and re-validate HNSW/BM25 indexes (`REINDEX` if the restore point predates the batch index build).

4.8.6 Recovery Targets and Failure Scenarios

Scenario	Target RPO	Target RTO	Path
Primary Postgres host failure	~0 (sync replica) / < 1 min (async)	15 min	Promote standby; repoint retrieval API and workers.
Logical corruption / bad ACL sync / operator error	≤ 1 hour	4 hours	PITR to the minute before the fault; §4.8.5 reconciliation.
Storage node failure (volume intact)	0	1 hour	Attach volume to standby storage node, re-export NFS, workers remount (hard mounts).
Volume loss or corruption	≤ 1 hour (backfill) / ≤ 24 hours (steady state)	8 hours	Restore latest verified snapshot in-AZ; §4.8.5 reconciliation; refetch delta from sources.

Scenario	Target RPO	Target RTO	Path
AZ / site loss	≤ 1 hour	8 hours	Restore Postgres from cross-AZ backup repository and volume from cross-AZ snapshot copy; full §4.8.5 sequence.
Ransomware / malicious deletion of backups	≤ 24 hours	24 hours	Locked snapshots and separate-account backup repository survive; restore from the oldest clean point identified by the scrub job and audit ledger.

4.8.7 Restore Testing, Monitoring, and Ownership

- **Monthly automated restore test:** restore the latest Postgres backup into an isolated instance, run `pg_checksums/amcheck`, execute the golden test suite (§4.7) against it, and verify the audit-ledger manifest chain. The test is a CI job; a failed restore pages the on-call engineer the same as a production outage.
- **Quarterly DR drill:** full §4.8.5 sequence from cross-AZ copies into a DR environment, timed against the RTO table above, with results reported to Compliance and Business Continuity.
- **Backup health alerts:** WAL-archive lag > 15 min, latest successful base backup older than 26 hours, snapshot copy failures, scrub mismatches, WORM export missing for a day, backup repository free space < 20%.
- **Ownership:** the platform team owns backup execution and restore drills; Compliance owns audit-archive retention rules; Business Continuity signs off on RPO/RTO targets and drill results. Run-books for each scenario in §4.8.6 are stored in the repository alongside the schema and reviewed after every drill.

5. Technology Choices

Layer	Selected Technology	Strategic Rationale
Originals & Derivatives	Mounted block/file volume — AWS EBS (gp3/io2) on a storage node exported via NFS v4, or on-prem NAS/SAN; XFS + LVM	Stays inside the firm's network with no object-store dependency; content-addressed sharded directory tree; write-once via atomic rename; snapshots for durability; local POSIX I/O for parsers; zero database bloat for binary payloads.
Parsing & Extraction	Docling (Primary) + PyMuPDF (Fast-path)	Accurate layout/table decomposition and OCR confidence scoring; fast-path accelerates born-digital PDFs by 10x.
Database & Indexing	PostgreSQL 16+ with <code>pgvector</code> & <code>pg_search</code>	Single engine for transactional consistency, ACL joins, BM25 text search, and vector similarity. Eliminates cross-system sync risks.
Vector Format	<code>halfvec</code> (16-bit FP16)	50% RAM savings with zero measurable drop in financial retrieval recall; 25M vectors fit in ~25 GB RAM.
Job Queue	Postgres <code>ingestion_jobs</code> via SKIP LOCKED	Native transactionality; zero external brokers (no Redis/RabbitMQ/Kafka required for v1).
Embeddings	Self-Hosted Open Model (e.g., BGE-Large / E5-v2 / Nomic)	Complete data boundary compliance; eliminates per-token API fees across 10B+ backfill tokens.

Layer	Selected Technology	Strategic Rationale
Reranker	BGE-Reranker-Large (Cross-Encoder on GPU)	Drastically improves precision for complex multi-hop financial clauses; evaluated conditionally.
Retrieval API	Python (FastAPI / asyncpg)	Lightweight, typed, low-overhead microservice interface decoupled from frontend agents.
LLM Inference	Enterprise Private Endpoint / Hosted vLLM	Strictly enclosed within internal VPC security perimeter.
Backup & Recovery	pgBackRest (or platform-managed PITR on RDS/Aurora) + streaming standby; EBS Data Lifecycle Manager snapshots with cross-AZ copy and snapshot lock (or array-native NAS snapshots/replication); WORM audit-log exports	Continuous WAL archiving delivers the 1-hour RPO; immutable, separately-scoped backup storage survives credential compromise; write-once volume makes snapshots consistent without quiescing; restore drills are automated so the recovery path is proven, not assumed.

6. Sizing, Performance, and Operational Notes

- **Corpus Sizing:**

- 3,000,000 source documents $\rightarrow$ ~45M raw chunks $\rightarrow$ **20M-25M indexed chunks** post-deduplication.
- Vector storage (25M chunks $\times$ 1024 dims $\times$ 2 bytes $\times$ 1.5x): **~50 GB** including HNSW graph index.
- Easily managed in a single PostgreSQL instance with 128 GB–256 GB RAM.

- **Ingestion Throughput & Job Table Management:**

- Dedicated worker pool utilizing PyMuPDF for born-digital files processes 50–100 pages/second per node.
- Scanned files on GPU-enabled Docling nodes process 8–15 pages/second.
- To prevent PostgreSQL catalog and vacuum degradation during initial backfill, `ingestion_jobs` logs are partitioned by date and aggressively pruned upon completion.

7. Scaling Gates (When to Evolve)

The chunk index can be transitioned to an external engine (e.g., OpenSearch or dedicated vector cluster) **if and only if** one of the following empirical thresholds is met:

1. Active indexed chunk count exceeds **50,000,000**.
2. Retrieval p95 latency under high concurrent load exceeds 2.0s despite index tuning and table partitioning.
3. Live write churn causes HNSW maintenance to noticeably degrade real-time read queries.

Because derivatives (Docling JSON and Markdown) are permanently stored on the document volume, migrating to an external search index is purely a background re-index task behind the FastAPI interface, requiring zero re-parsing of raw documents.

A separate storage gate applies to the volume itself: if the corpus approaches ~14 TiB on a single gp3 volume, or the single NFS storage node becomes an I/O bottleneck during backfill, add volumes under LVM, split the tree across several exports by hash prefix, or move to a managed NFS service. None of these change the directory layout or the URIs stored in PostgreSQL.

8. Delivery Roadmap



9. Risk Matrix & Mitigations

Risk Scenario	Impact	Engineered Mitigation
Permission Leakage	Critical	Pre-filtering in SQL with hard join on <code>acl_entries</code> ; automated permissions testing in CI/CD eval suite.
Backfill Index Churn	High	Defer HNSW/BM25 index generation until after bulk chunk insertion; prune completed ingestion jobs immediately.
Missing Specific Clauses	High	Adaptive routing: queries with exact identifiers bypass doc-summary filters directly to the chunk index.

Risk Scenario	Impact	Engineered Mitigation
Embedding Model Drift	Medium	Record <code>embedding_model</code> version per chunk; support blue/green parallel indices for zero-downtime model upgrades.
Hallucinated Citations	High	Enforce strict citation syntax validation in retrieval middleware; reject answers lacking verified chunk ID tokens.
Document Volume Loss or Corruption	Critical	Scheduled EBS/NAS snapshots replicated to a second AZ/site; write-once atomic-rename discipline; periodic hash scrub against <code>document_versions.content_hash</code> .
Storage Node as Single Point of Failure	High	Standby storage node with snapshot restore runbook; NFS mounts configured with hard retries; ingestion jobs are idempotent and resume after remount.
PostgreSQL Loss or Logical Corruption	Critical	Continuous WAL archiving + daily incremental / weekly full base backups (PITR, RPO ≤ 1 h); streaming standby for host failure; monthly automated restore test with golden-suite validation (§4.8.2, §4.8.7).
Audit Record Loss (regulatory)	Critical	Monthly-partitioned <code>query_audit_logs</code> exported nightly to write-once storage with hash manifests; retention governed by records policy, not engineering jobs (§4.8.3).
Database-Volume Inconsistency After Restore	High	Defined restore ordering (DB to point T, volume snapshot $\geq$ T); reconciliation job re-hashes referenced files and queues refetch for any gaps; ACL re-sync gate before reopening retrieval (§4.8.5).
Backup Destruction (ransomware, compromised admin)	Critical	Snapshot lock / recycle-bin retention; backup repository in a separate account or array; WORM audit archive; restore from oldest clean point verified by scrub and ledger (§4.8.4, §4.8.6).
Untested Restore Path	High	Restore tests are CI jobs that page on failure; quarterly DR drills timed against RTO targets and reported to Compliance/BC (§4.8.7).

10. Summary

This v3 architecture maximizes operational simplicity and cost-efficiency by leveraging PostgreSQL for relational metadata, ACL enforcement, full-text BM25 search, and vector search in a single unified cluster, with originals and derivatives held on mounted block/file volumes inside the firm's own network. By decoupling raw parsing from search indexing, optimizing bulk-load procedures, and establishing adaptive hybrid retrieval, the system delivers high accuracy and compliance without unnecessary infrastructure complexity. v4 closes the recovery gap: point-in-time database backups, immutable audit-log archives, locked volume snapshots, a defined restore order, and scheduled restore drills give the platform an RPO of one hour and a proven path back to service.